



# Lane Keep Assist (LKA) Research: Architecture & Implementation Strategy





# Lane Detection Models: Technical Overview

## 1. Ultra-Fast-Lane-Detection (UFLD) v2

This model does not attempt to "draw" the line pixel-by-pixel; instead, it divides the image into a grid and selects which cell the line passes through (Row-based classification).

### Pros:

- **Extreme Speed:** Easily reaches 300+ FPS. On the Hailo-8, latency will be near zero.
- **Lightweight:** Occupies very little memory/bandwidth on the Raspberry Pi 5 PCIe bus.
- **Simplicity:** The output consists of direct (x,y) coordinates, simplifying curve calculation in OpenCV.

### CONS:

- **Rigidity:** May struggle with extremely sharp curves (90-degree angles) if the grid density is insufficient.
- **Perspective Sensitivity:** If the camera is mounted too low, the perspective can confuse the classification of distant lines.

# Lane Detection Models: Technical Overview

## 2. YOLOP (You Only Look Once for Panoptic Driving)

An "all-in-one" model that performs three tasks simultaneously: object detection (cars/cones), drivable area segmentation, and lane detection.

### Pros:

- **Resource Efficiency:** Instead of running three separate models, you run only one on the Hailo NPU, saving power and bandwidth.
- **Contextual Awareness:** By detecting the "drivable area," the LKA system becomes more robust (it "knows" the lane must be at the edge of this area).
- **Hailo Support:** An optimized version is already available in the Hailo Model Zoo.

### CONS:

- **Heavy Compute:** Slower than UFLD. You might need to reduce the input resolution.
- **Post-processing Complexity:** Extracting lines from a segmentation mask requires more CPU processing on the RPi 5.



# Lane Detection Models: Technical Overview

## 3. LaneATT (Anchor-based Lane Detection)

Uses the concept of "anchors" (pre-defined lines) and attempts to pull/adjust them to fit the actual image. It represents a balance between the previous two models.

### Pros:

- **Curve Precision:** Excellent at detecting lanes that disappear into the horizon or sharp turns.
- **Resilience:** Handles occlusions very well (e.g., when an obstacle partially covers the line).

### CONS:

- **Calibration Dependent:** Highly dependent on camera position. If the height is adjusted manually, the "anchors" may require re-tuning.
- **Conversion Challenges:** Converting to the Hailo .hef format can be more difficult if the model uses operations not natively supported by the Dataflow Compiler.

# Lane Detection Models: Technical Overview

| Criterion                         | UFLD v2       | YOLOP           | LaneATT             |
|-----------------------------------|---------------|-----------------|---------------------|
| FPS<br>(Estimated on Hailo-8)     | > 200         | ~ 40-60         | ~ 80-100            |
| Integration Complexity            | Low           | High            | Medium              |
| Robustness<br>(Shadows/Occlusion) | Medium        | High            | High                |
| CPU Usage<br>(RPi 5)              | Minimal       | High            | Medium              |
| Best for...                       | Maximum Speed | Full ADAS Suite | Geometric Precision |



# Steering Control: Technical Overview

## 1. Pure Pursuit (The Standard for PiRacer)

A geometric method that calculates the curvature required for the car to reach a "Target Point" (Look-ahead point) on the lane's centerline.

**How it works:** Imagine the car is chasing a "carrot" placed at a distance  $L_{fd}$  in front of it on the ideal trajectory.

### Pros:

- **Stability:** Highly robust against noise in lane detection.
- **Simplicity:** Easy to implement in Python/C++ with very low CPU consumption.
- **Intuitive Tuning:** You only need to adjust one main parameter (the look-ahead distance).

### CONS:

- **Corner Cutting:** If the look-ahead distance is too large, the car will "cut" the corners.
- **Rigidity:** Does not account for vehicle skid or road bank/inclination.



# Steering Control: Technical Overview

## 2. Stanley Controller

Unlike Pure Pursuit, the Stanley controller focuses on the position error at the front axle and the orientation error (heading error).

**How it works:** It combines the lateral track error ( $e_y$ ) with the heading error ( $\theta$ ) relative to the lane.

### Pros:

- **Precision:** Converges to the lane center much faster than Pure Pursuit.
- **Smoothness:** Provides more natural steering adjustments, preventing "snaking" on straightaways.

### CONS:

- **Noise Sensitivity:** If lane detection fails for even a millisecond, the Stanley controller can cause a sudden steering "jerk."
- **Complexity:** Requires finer calibration.



# Steering Control: Technical Overview

## 3. MPC (Model Predictive Control) - The "State of the Art"

This algorithm uses a mathematical model of the car (mass, tire friction, moment of inertia) to predict future behavior.

**How it works:** It solves a real-time optimization problem to find the trajectory that minimizes error and energy consumption over the next few seconds.

### Pros:

- **Supreme Performance:** Enables autonomous driving at high speeds or in edge-case situations.
- **Safety:** Can incorporate hard constraints (e.g., "never turn the steering wheel more than 30 degrees").

### CONS:

- **Computational Cost:** Very heavy for a Raspberry Pi 5 if the prediction horizon is long.
- **Difficulty:** Requires advanced knowledge of linear algebra and optimization.



# Lane Detection Models: Technical Overview

| Algorithm    | Complexity | Stability | Accuracy | Best For                       |
|--------------|------------|-----------|----------|--------------------------------|
| Pure Pursuit | Low        | High      | Medium   | Our MVP / PiRacer              |
| Stanley      | Medium     | Medium    | High     | High-speed track racing        |
| MPC          | High       | High      | Supreme  | Professional self-driving cars |



# Thank you!

drivapi

